

STAT 3255: Midterm

Scott McCabe

Table of contents

1	Take-Home Midterm Exam	1
1.1	15.1 Overview	1
1.2	15.1.1 Data Source	2
1.3	15.1.2 Problem	2
1.4	15.1.3 Expectations for your submission	2
1.5	15.1.3.1 Load packages and data	3
1.6	15.2 Part 1 - Data import and data quality	3
	1.6.1 15.2.1 Task 1 - Data checks: missingness, duplicates, consistency	3
	1.6.2 15.2.2 Task 2 - Examine individual variables and marginal distributions	4
1.7	15.3 Part 2 - Exploratory data analysis (EDA)	5
	1.7.1 Task 3 - Summary statistics for the response (demand)	5
	1.7.2 15.3.2 Task 4 - Visualize Trends over time	6
	1.7.3 15.3.3 Task 5 - Visual comparisons by groups	7
1.8	15.4 - Part 3 - Predictive Modeling	9
	1.8.1 15.4.1 Task 6 - Set up modeling target and features	9
	1.8.2 15.4.2 Task 7 - Train/test split	10
	1.8.3 15.4.3 Task 8 - Model Training	10
1.9	15.5 Part 4 - Summary	14
	1.9.1 15.5.1 Task 9	14

1 Take-Home Midterm Exam

1.1 15.1 Overview

In this take-home exam, you will work with the Bike Sharing Dataset (daily data, day.csv) to explore and predict bike rental demand. The exam is designed to test your skills in data import, cleaning, exploratory data analysis (EDA), and predictive modeling. You will also write a short memo summarizing your findings and recommendations.

1.2 15.1.1 Data Source

The dataset comes from the UCI Machine Learning Repository: Bike Sharing Dataset by Hadi Fanaee-T. It contains daily counts of bike rentals in Washington, D.C. along with weather and calendar information.

You will use `day.csv` (731 rows; one row per day). Variables (from the dataset README) are listed below:

- `dteday`: date
- `season`: season (1=spring, 2=summer, 3=fall, 4=winter)
- `yr`: year (0=2011, 1=2012)
- `mnth`: month (1–12)
- `holiday`: whether the day is a holiday (1=yes, 0=no)
- `weekday`: day of week (0–6)
- `workingday`: 1 if neither weekend nor holiday, else 0
- `weathersit`:
 - 1: Clear / few clouds / partly cloudy
 - 2: Mist + cloudy
 - 3: Light snow / light rain + thunderstorm
 - 4: Heavy rain / ice pellets / snow + fog
- `temp`: normalized temperature in Celsius (divided by 41)
- `atemp`: normalized “feels like” temperature (divided by 50)
- `hum`: normalized humidity (divided by 100)
- `windspeed`: normalized wind speed (divided by 67)
- `casual`: count of casual users
- `registered`: count of registered users
- `cnt`: total rentals (casual + registered)

1.3 15.1.2 Problem

Our main goal is to forecast daily bike rental demand (`cnt`). Accurate forecasts can help staffing, bike rebalancing, and maintenance planning.

1.4 15.1.3 Expectations for your submission

- You should submit:
 - A `_exam_bike_name.qmd` file of your report with your code/answers embedded. Your code should be executable and produce the same results when run.
 - A rendered HTML or PDF report. The report should be well-organized and clearly labeled.
- You may follow the steps and use any reasonable statistical/machine learning/AI approaches.

- Your answers should be reproducible.
- You should justify key choices if applicable (e.g., data splitting, feature engineering, metric selection, model choice).

1.5 15.1.3.1 Load packages and data

Load the packages you plan to use (e.g., pandas, numpy, matplotlib or seaborn, and scikit-learn). For example:

```
import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
# import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

You can also import `sklearn.compose.ColumnTransformer`, `sklearn.preprocessing.OneHotEncoder`, and models like `LinearRegression`, `Ridge`, `RandomForestRegressor`, etc.

Place `day.csv` in the same folder as your `.qmd`, or in a `data/` subfolder.

```
#load data
day = pd.read_csv("day.csv")
day["dteday"] = pd.to_datetime(day["dteday"])
```

```
day.shape
```

```
(731, 16)
```

1.6 15.2 Part 1 - Data import and data quality

1.6.1 15.2.1 Task 1 - Data checks: missingness, duplicates, consistency

- Display the first 5-10 rows of the dataset to understand its structure.
- How many missing values are in each column?
- Are there duplicated dates (`dteday`)? If yes, how many?
- Verify whether `cnt` equals `casual` + `registered` for all rows. If not, report how many rows fail.

```
day.head(10)
```

```
missing_values = day.isnull().sum()
print(f"Missing values in each column: {missing_values}")
```

```

duplicate_dates = day["dteday"].duplicated().sum()
print(f"Number of duplicated dates: {duplicate_dates}")

cnt_mismatch = (day["cnt"] != (day["casual"] + day["registered"]))
mismatch_count = cnt_mismatch.sum()
print(f"Number of rows where cnt does not equal casual + registered: {mismatch_count}")

```

```

Missing values in each column: instant      0
dteday      0
season      0
yr          0
mnth       0
holiday     0
weekday     0
workingday  0
weathersit   0
temp        0
atemp       0
hum         0
windspeed   0
casual      0
registered  0
cnt         0
dtype: int64
Number of duplicated dates: 0
Number of rows where cnt does not equal casual + registered: 0

```

- Summary: There are no missing values in any of the columns. There are no duplicated dates in the dataset. There are no rows where cnt does not equal casual + registered.

1.6.2 15.2.2 Task 2 - Examine individual variables and marginal distributions

- Report summary statistics of temp, atemp, hum, windspeed, or make some visualization (e.g., boxplot, histogram) of those variables. Note that these variables are normalized to be between 0 and 1 (except that humidity can be 0).
- Briefly comment: do these distributions look plausible? Any potential outliers or anomalies?

Summary Statistics:

```

weather_vars = ["temp", "atemp", "hum", "windspeed"]
day[weather_vars].describe()

```

	temp	atemp	hum	windspeed
count	731.000000	731.000000	731.000000	731.000000

	temp	atemp	hum	windspeed
mean	0.495385	0.474354	0.627894	0.190486
std	0.183051	0.162961	0.142429	0.077498
min	0.059130	0.079070	0.000000	0.022392
25%	0.337083	0.337842	0.520000	0.134950
50%	0.498333	0.486733	0.626667	0.180975
75%	0.655417	0.608602	0.730209	0.233214
max	0.861667	0.840896	0.972500	0.507463

Summary statistics temp: count = 731 mean = 0.495385 std = 0.183051 min = 0.059130 25% = 0.337083 50% = 0.498333 75% = 0.655417 max = 0.861667

atemp: count = 731 mean = 0.474354 std = 0.162961 min = 0.079070 25% = 0.337842 50% = 0.486733 75% = 0.608602 max = 0.840896

hum: count = 731 mean = 0.627894 std = 0.142429 min = 0.000000 25% = 0.520000 50% = 0.626667 75% = 0.730209 max = 0.972500

windspeed: count = 731 mean = 0.190486 std = 0.077498 min = 0.022392 25% = 0.134950 50% = 0.180975 75% = 0.233214 max = 0.507463

These distributions all look plausible. For the two temperature variables, the mean and median are around 0.5, which makes sense given average temperatures in Washington, D.C. The humidity variable has a mean around 0.63, which also seems reasonable. The windspeed variable has a mean around 0.19, which is reasonable for daily average wind speeds. There do not appear to be any extreme outliers in these variables.

1.7 15.3 Part 2 - Exploratory data analysis (EDA)

1.7.1 Task 3 - Summary statistics for the response (demand)

Compute and report:

- Overall mean, median, and standard deviation of cnt.
- Mean cnt by year (yr), and the difference between 2012 and 2011.

Briefly interpret what you see.

```
cnt_mean = day["cnt"].mean()
cnt_median = day["cnt"].median()
cnt_std = day["cnt"].std()
```

```
print(f"Overall mean of cnt: {cnt_mean}")
print(f"Overall median of cnt: {cnt_median}")
print(f"Overall standard deviation of cnt: {cnt_std}")
```

```

mean_by_year = day.groupby("yr")["cnt"].mean()
print(f"Mean cnt by year:\n{mean_by_year}")

difference = mean_by_year[1] - mean_by_year[0]
print(f"Difference in mean cnt between 2012 and 2011: {difference}")

```

```

Overall mean of cnt: 4504.3488372093025
Overall median of cnt: 4548.0
Overall standard deviation of cnt: 1937.2114516187676
Mean cnt by year:
yr
0    3405.761644
1    5599.934426
Name: cnt, dtype: float64
Difference in mean cnt between 2012 and 2011: 2194.172782393892

```

- Interpretation: There was a massive increase in bike rentals from 2011 to 2012. The mean cnt in 2011 was approximately 3406, while the mean cnt in 2012 was approximately 5600. This is an increase of about 2196 rentals per day on average.

1.7.2 15.3.2 Task 4 - Visualize Trends over time

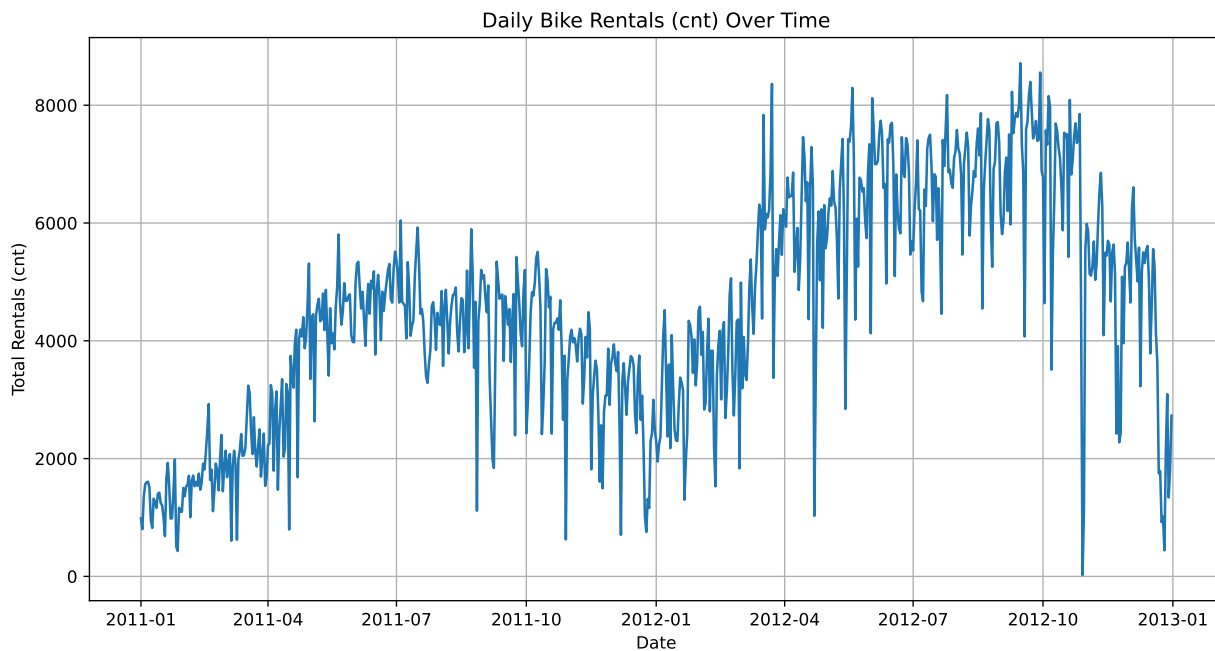
Make a time-series plot of cnt vs dteday.

- Is there an overall trend over time?
- Do you see seasonality (repeating yearly patterns)?

```

plt.figure(figsize=(12, 6))
plt.plot(day["dteday"], day["cnt"])
plt.title("Daily Bike Rentals (cnt) Over Time")
plt.xlabel("Date")
plt.ylabel("Total Rentals (cnt)")
plt.grid()
plt.show()

```



- Is there an overall trend over time? There does seem to be an overall increase in bike rentals over the two-year span, with rental counts going up during the warmer months and down during the colder months. The increase from 2011 to 2012 is also evident in the plot.
- Do you see seasonality (repeating yearly patterns)? Yes. There is clear seasonality in data, with the peaks being during the late summer, and the lowest points being during the winter. This pattern is evident in both years.

1.7.3 15.3.3 Task 5 - Visual comparisons by groups

Create two appropriate and informative plots (your choice) to compare cnt across groups. Pick from:

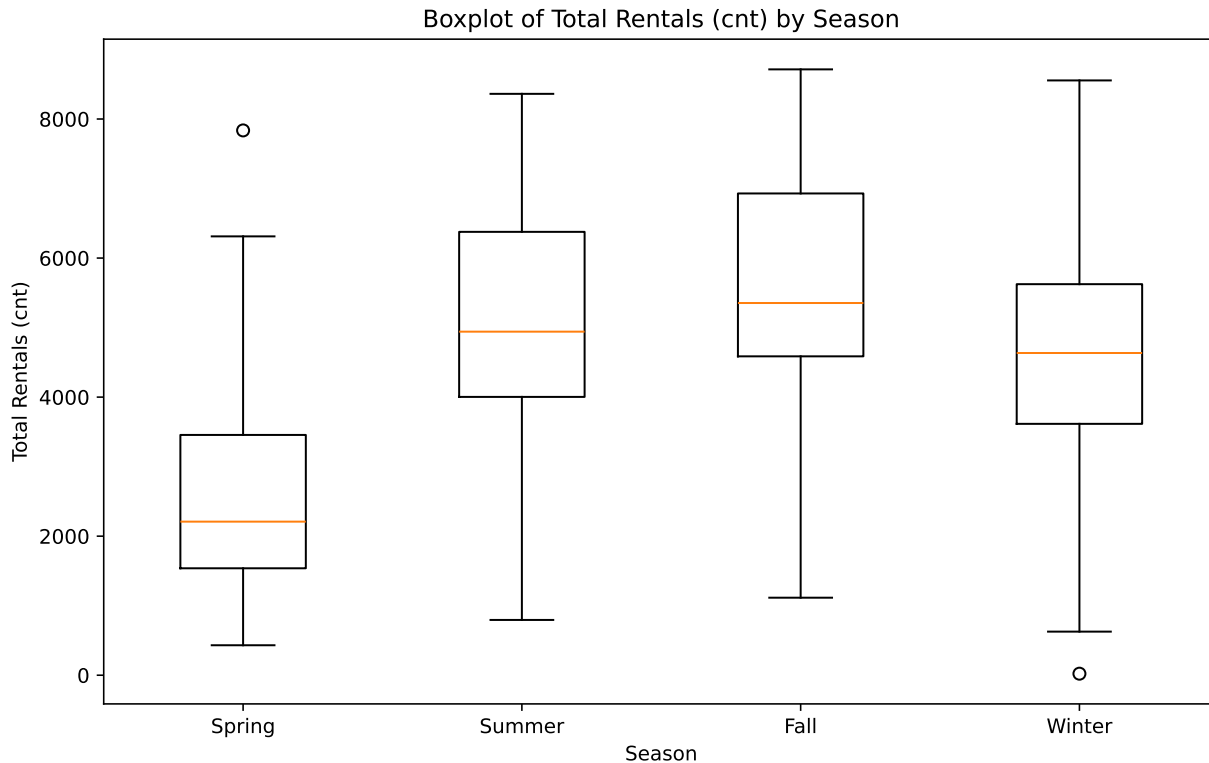
- season
- weathersit
- workingday
- weekday
- mnth

Write a short interpretation for each plot.

```
seasons = [1,2,3,4]
season_names = ["Spring", "Summer", "Fall", "Winter"]
data_by_season = [day.loc[day["season"] == season, "cnt"] for season in seasons]

plt.figure(figsize=(10, 6))
plt.boxplot(data_by_season, labels=season_names)
```

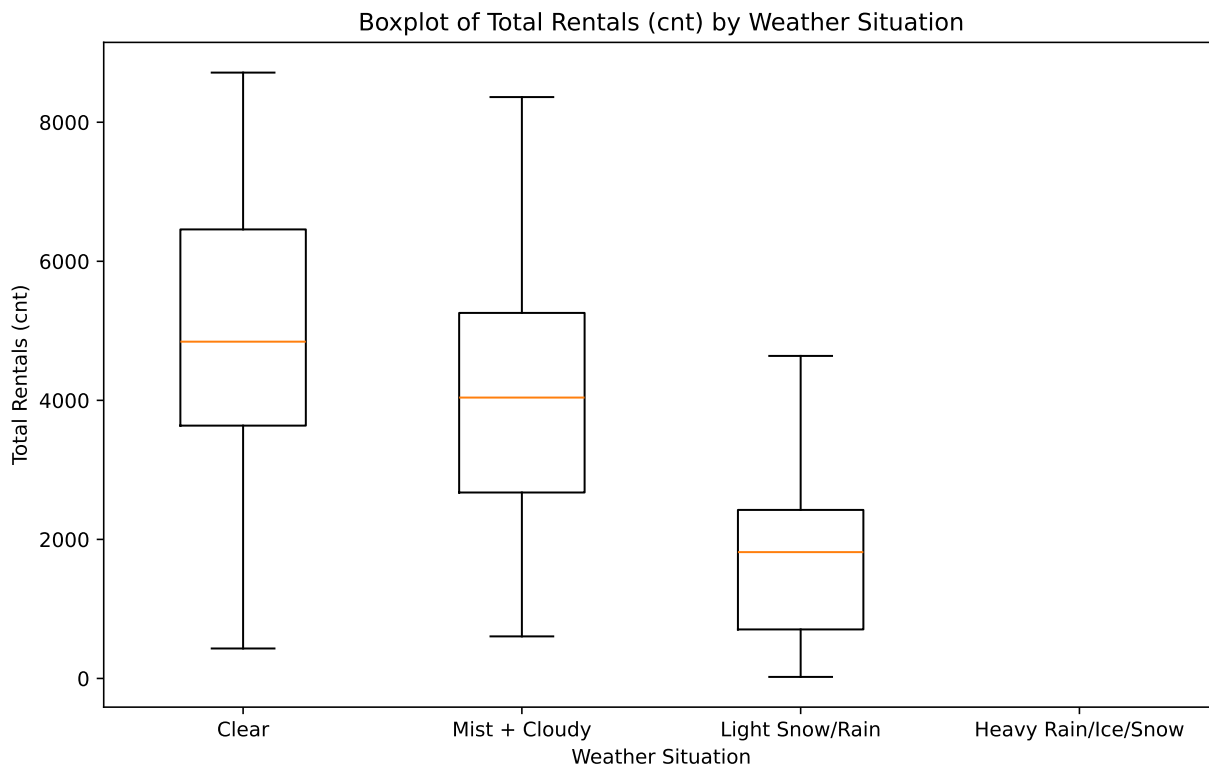
```
plt.title("Boxplot of Total Rentals (cnt) by Season")
plt.xlabel("Season")
plt.ylabel("Total Rentals (cnt)")
plt.show()
```



- Interpretation for plot of mean cnt grouped by season: Summer, Fall, and Winter have similar median rental counts, while Spring has a noticeably lower median. This makes sense given the time series plot, which showed peaks in the warmer months and low points in the cold months. It is surprising though that winter has a higher median than spring. This may be because rentals are still somewhat high at the beginning of the Winter, but taper off by the end of the season.

```
weather_codes = [1, 2, 3, 4]
weather_names = ["Clear", "Mist + Cloudy", "Light Snow/Rain", "Heavy Rain/Ice/Snow"]
data_by_weather = [day.loc[day["weathersit"] == code, "cnt"] for code in weather_codes]

plt.figure(figsize=(10, 6))
plt.boxplot(data_by_weather, labels=weather_names)
plt.title("Boxplot of Total Rentals (cnt) by Weather Situation")
plt.xlabel("Weather Situation")
plt.ylabel("Total Rentals (cnt)")
plt.show()
```



- Interpretation of plot of mean cnt grouped by weathersit: The box plot shows that clear weather has the highest median rental count, followed by mist/cloudy, then light snow/rain. There is no data shown for heavy rain/ice/snow, which suggests that nobody rented bikes on these days. These results makes sense, as the better the weather, the more likely people are to rent bikes.

1.8 15.4 - Part 3 - Predictive Modeling

1.8.1 15.4.1 Task 6 - Set up modeling target and features

We will predict: * Target: cnt (daily total rentals)

Choose feature set X using only variables that would be known at prediction time. For example:

```
features = ["season", "yr", "mnth", "holiday", "weekday", "workingday",
            "weathersit", "temp", "atemp", "hum", "windspeed"]
X = day[features].copy()
y = day["cnt"].copy()
```

- Explain briefly (1–3 sentences) why you included these features and excluded others. In particular, why casual and registered should be excluded.
- If you want to engineer new features, briefly explain your rationale.

I included the features from the template, as they are all either calendar variables (season, yr, mnth, holiday, weekday, workingday) or weather variables (weathersit, temp, atemp, hum, windspeed) that would be known at the time of prediction. I excluded casual and registered because they sum up to the target variable cnt. Including them would leak future information into the model, which would lead to unrealistic predictions. I chose not to engineer any new features, as the features we already have are sufficient for a basic predictive model. Engineering new features would only overcomplicate the model in my opinion.

1.8.2 15.4.2 Task 7 - Train/test split

We will use the following code to make train/test data split.

```
day_sorted = day.sort_values("dteday").reset_index(drop=True)
```

```
train_prop = 0.80
```

```
split = int(len(day_sorted) * train_prop)
```

```
train = day_sorted.iloc[:split]
```

```
test = day_sorted.iloc[split:]
```

```
X_train = train[features]
```

```
y_train = train["cnt"]
```

```
X_test = test[features]
```

```
y_test = test["cnt"]
```

```
len(train), len(test)
```

```
(584, 147)
```

- Explain how the train/test split is done in the provided code.

The data is sorted by date, and then the first 80% of the data is used for training, while the remaining 20% is used for testing. This means that the model is trained on the earlier dates and tested on the later dates.

- Explain why we do not use random splitting.

Random splitting would mix up the order of the data, which could cause past data to be learning from future data. This would lead to unrealistic performance and data leakage.

1.8.3 15.4.3 Task 8 - Model Training

The following code fits a ridge regression with one-hot encoding for categorical variables.

```

import numpy as np
import pandas as pd

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import Ridge
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import mean_squared_error, r2_score

# -----
# Assumes you already have: X_train, y_train, X_test, y_test
# -----

# 1) Identify categorical vs numeric columns
# Treat integer-coded calendar/weather fields as categorical if present
cat_cols = []
for c in ["season", "yr", "mnth", "holiday", "weekday", "workingday", "weathersit"]:
    if c in X_train.columns:
        cat_cols.append(c)

# Also include any object/category columns (e.g., if you kept dteday as string)
cat_cols += X_train.select_dtypes(include=["object", "category"]).columns.tolist()
cat_cols = sorted(set(cat_cols))

num_cols = [c for c in X_train.columns if c not in cat_cols]

# 2) Preprocess: scale numeric, one-hot categorical
preprocess = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), num_cols),
        ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols),
    ],
    remainder="drop",
)

# 3) Ridge pipeline + CV tuning
ridge_pipe = Pipeline(
    steps=[
        ("preprocess", preprocess),
        ("model", Ridge())
    ]
)

param_grid = {

```

```

    "model__alpha": np.logspace(-3, 3, 25) # tune strength of regularization
}

grid = GridSearchCV(
    estimator=ridge_pipe,
    param_grid=param_grid,
    scoring="neg_root_mean_squared_error",
    cv=5,
    n_jobs=-1,
)

grid.fit(X_train, y_train)

print("Best alpha:", grid.best_params_["model__alpha"])
best_ridge = grid.best_estimator_

# 4) Evaluate on test set
pred = best_ridge.predict(X_test)
mse = mean_squared_error(y_test, pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, pred)

print(f"Test RMSE: {rmse:,.2f}")
print(f"Test R^2 : {r2:.3f}")

Best alpha: 1.7782794100389228
Test RMSE: 1,105.08
Test R^2 : 0.652

```

- Use the same train/test split, fit another model of your choice. You may optionally consider other ways of possibly improving the model, e.g., variable transformation, feature engineering, etc.
- Evaluate your model with the same metrics. Compare performance and state which model you would choose.
- (Bonus) Your model does not need to outperform the ridge regression, but if it does, you get bonus points.
- (Bonus) Report the top-5 features from either models based on feature importance metrics, and briefly interpret them.

New model: Random Forest from sklearn

```

from sklearn.ensemble import RandomForestRegressor

rf_pipe = Pipeline(
    steps=[

```

```

        ("preprocess", preprocess),
        ("model", RandomForestRegressor(n_estimators=200, random_state=42))
    ]
)

rf_pipe.fit(X_train, y_train)
rf_pred = rf_pipe.predict(X_test)
rf_mse = mean_squared_error(y_test, rf_pred)
rf_rmse = np.sqrt(rf_mse)
rf_r2 = r2_score(y_test, rf_pred)

print(f"Random Forest Test RMSE: {rf_rmse:,.2f}")
print(f"Random Forest Test R^2 : {rf_r2:.3f}")

```

Random Forest Test RMSE: 1,248.19
Random Forest Test R² : 0.557

- Summary of the Random Forest model: The random forest model proudced a test RMSE of 1,248.19 and a test R² of 0.557, compared to the ridge regression model which has a test RMSE of 1,105.08 and a test R² of 0.652. I would pick the ridge regression model, as it has better performance on the test set.
- Getting Top N Features for Random Forest (Bonus):

```

onehot = rf_pipe.named_steps["preprocess"].named_transformers_["cat"]
cat_feature_names = onehot.get_feature_names_out(cat_cols)
feature_names = num_cols + list(cat_feature_names)

importances = rf_pipe.named_steps["model"].feature_importances_
feature_importance_df = pd.DataFrame({
    "feature": feature_names,
    "importance": importances
}).sort_values("importance", ascending=False)

feature_importance_df.head(5)

```

	feature	importance
1	atemp	0.283362
0	temp	0.276676
35	yr_1	0.135155
34	yr_0	0.111592
2	hum	0.060782

- Top 5 Features:

1. atemp - importance = 0.283362
2. temp - importance = 0.276676
3. yr_1 - importance = 0.135155
4. yr_2 - importance = 0.111592
5. hum - importance = 0.060782

1.9 15.5 Part 4 - Summary

1.9.1 15.5.1 Task 9

Write a short memo (as if to a bike-share operations manager).

- 2–3 key insights from your analysis
- What model you chose and why
- One or two limitations and one or two next steps (e.g., adding external features or handling holidays better)
 - Summary/Short Memo: My analysis of the bike sharing dataset revealed several key insights. First, there was a significant increase in the number of bike rentals from 2011 to 2012, which suggests that bike sharing is growing in popularity. This is a great sign! Second, there is a clear relationship between weather and bike rentals, with more rentals occurring on clear days and fewer on days with bad weather. This is reflected in the correlation with season as well, with more rentals in the warmer months. For predicting rentals, I would recommend using the ridge regression model, as it had better performance compared to the Random Forest model. Looking forward to the future, there are some limitations with this data set. The data only covers two years, which may not be enough to see longer-term trends. Also, the data does not include any sort of tracking of particular special events that may have affected bike rentals. I would recommend collecting more data over a longer period, and adding a variable that captures whether there was a special event on a specific day, such as a parade or a sports game.